Data mining et apprentissage automatique appliqué

Informatique — Licence 3 (L3)

Cours INF-L3-17

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Passer de la théorie du machine learning à la pratique complète : préparer des données réelles, entraîner arbres et forêts aléatoires, réduire les dimensions, prévoir des séries temporelles, recommander, déployer un modèle en API et le surveiller en production.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Préparation des données : nettoyage et encodage
• Leçon 2 — Arbres de décision et forêts aléatoires
• Leçon 3 — Réduction de dimension et sélection de variables
• Leçon 4 — Séries temporelles : prévisions simples
• Leçon 5 — Systèmes de recommandation : initiation
• Leçon 6 — Déployer un modèle : API et suivi des performances
• Leçon 7 — Projet : prédiction à partir de données réelles
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En L2 et au début de la L3, tu as découvert les principes de l'apprentissage automatique : un modèle apprend à partir d'exemples, puis prédit sur des cas nouveaux. Ce cours change d'échelle : il s'agit maintenant d'appliquer ces méthodes comme le ferait un professionnel, sur des données imparfaites, avec des contraintes de temps, et en assumant les résultats devant un utilisateur réel. Le data mining — la fouille de données — est exactement cela : transformer des données brutes en décisions, en suivant une chaîne complète et rigoureuse. C'est le quotidien des data scientists, des analystes et des développeurs qui, dans les entreprises comme dans les administrations, doivent prédire une demande, détecter une anomalie, recommander un produit ou anticiper un risque à partir de l'historique disponible.

Chaque leçon suit le même fil : un problème concret, un petit jeu de données construit pour le cours (ou un jeu public intégré à scikit-learn, nommé exactement), du code Python exécuté pour de vrai, et des résultats chiffrés que tu pourras reproduire à l'identique. Les chiffres de ce cours ne sont pas des illustrations inventées : ils sortent des exécutions présentées, avec leurs imperfections — un modèle à 0,80 d'exactitude, un autre qui se trompe une fois sur deux sur les admis. C'est en regardant ces imperfections en face qu'on apprend le métier. Tu es donc invité à réinstaller le même environnement (Python, pandas, scikit-learn), à recopier chaque exemple et à modifier les données pour observer comment les résultats réagissent : c'est par cette expérimentation guidée que les réflexes du praticien se forment.

Un avertissement traverse tout le cours : le piège de la fuite de données (data leakage). Si une information du futur ou du jeu de test influence l'entraînement, les scores deviennent mensongers — parfois spectaculaires, toujours faux. Nous nommerons ce piège à chaque étape : séparation des données, encodage, normalisation, sélection de variables, validation des séries temporelles. Retenir où il se cache vaut plus que n'importe quel algorithme. De même, le cours insiste sur la fin de la chaîne, trop souvent négligée : un modèle ne vaut que déployé, surveillé, et capable d'avouer quand le monde a changé autour de lui. Préparer, modéliser, évaluer honnêtement, déployer, surveiller : telle est la boucle complète que tu sauras dérouler à la fin de ce cours, et qui servira de trame au projet final sur données réelles.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Nettoyer un jeu de données (doublons, valeurs manquantes, valeurs aberrantes) et encoder ses variables catégorielles sans fuite de données.
- Entraîner un arbre de décision et une forêt aléatoire, puis interpréter leurs importances de variables et leurs erreurs.
- Réduire le nombre de variables par analyse en composantes principales (ACP) et par sélection statistique.
- Construire des prévisions simples sur une série temporelle (naïve, moyenne mobile, saisonnière) et les évaluer sans mélanger passé et futur.
- Calculer une recommandation par filtrage collaboratif fondé sur la similarité cosinus.
- Déployer un modèle derrière une API et mettre en place un suivi de ses performances (dérive des données et de la relation).
- Mener un projet de prédiction complet sur un jeu de données réel, de la question métier au bilan honnête des résultats.

Leçon 1 — Préparation des données : nettoyage et encodage

Aucun modèle ne rattrape des données sales. La préparation suit presque toujours le même ordre : d'abord comprendre le problème et regarder les données (combien de lignes ? quelles colonnes ? quels types ?), puis supprimer les doublons exacts, traiter les valeurs manquantes, repérer les valeurs aberrantes, et seulement ensuite transformer les variables pour les rendre utilisables par un algorithme. Dans notre exemple, le journal de commandes d'une petite boutique en ligne contient 12 lignes : un doublon parfait (la commande 8 saisie deux fois), une quantité manquante et un prix manquant. Après suppression du doublon, il reste 11 lignes propres à exploiter.

Pour les valeurs manquantes, trois stratégies dominent : supprimer les lignes concernées (acceptable si elles sont rares), remplacer par une statistique simple comme la médiane — plus robuste que la moyenne face aux valeurs extrêmes — ou créer une catégorie « inconnu » pour les variables textuelles. Le point critique est le moment où l'on calcule cette médiane : elle doit être calculée sur le seul jeu d'entraînement, après la séparation entraînement/test. Dans notre exemple, la

médiane des prix vaut 600 FCFA sur l'entraînement, mais 525 FCFA si on la calcule sur tout le jeu : utiliser 525, c'est laisser le jeu de test « parler » à l'entraînement — une fuite de données discrète mais réelle, qui fausse ensuite l'évaluation.

Les algorithmes de scikit-learn ne lisent que des nombres : les variables catégorielles (ville, mode de paiement) doivent être encodées. L'encodage one-hot crée une colonne binaire par catégorie : Bamako, Kayes et Sikasso donnent trois colonnes, et les trois modes de paiement trois autres, soit six colonnes au total. L'encodeur s'ajuste (fit) sur l'entraînement seul, pour la même raison que la médiane. Le paramètre handle_unknown='ignore' protège la production : si une ville jamais vue arrive ensuite (par exemple Ségou), la ligne est encodée avec des zéros partout plutôt que de faire planter le programme. Notre matrice d'entraînement finale compte 8 lignes et 6 colonnes, celle de test 3 lignes et 6 colonnes : mêmes colonnes, dans le même ordre — c'est la condition pour que le modèle comprenne les données.

L'essentiel
Ordre de préparation : regarder, dédoublonner, séparer entraînement/test, PUIS calculer médianes et ajuster les encodeurs sur l'entraînement seul. Toute statistique calculée sur tout le jeu avant la séparation est une fuite de données.

Exemple — Nettoyer et encoder les commandes d'une boutique (résultats réels)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

colonnes = ["id", "ville", "quantite", "prix", "paiement"]
lignes = [
[1, "Bamako", 5, 450, "Orange Money"],
[2, "Sikasso", 2, 1200, "Wave"],
[3, "Bamako", None, 600, "Especes"],
[4, "Kayes", 3, 450, "Orange Money"],
[5, "Bamako", 1, None, "Wave"],
[6, "Sikasso", 4, 450, "Orange Money"],
[7, "Bamako", 2, 600, "Wave"],
[8, "Kayes", 2, 1200, "Especes"],
[8, "Kayes", 2, 1200, "Especes"],
[9, "Bamako", 6, 450, "Wave"],
[10, "Sikasso", 3, 600, "Orange Money"],
[11, "Kayes", 2, 450, "Wave"],
]
df = pd.DataFrame(lignes, columns=colonnes)
print("lignes brutes :", len(df)) # 12
df = df.drop_duplicates()
print("apres doublons :", len(df)) # 11
# Separation AVANT tout calcul de statistique
train, test = train_test_split(df, test_size=0.25, random_state=42)
med_q = train["quantite"].median() # 2.0
med_p = train["prix"].median() # 600.0
for d in (train, test):
d["quantite"] = d["quantite"].fillna(med_q)
d["prix"] = d["prix"].fillna(med_p)
enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
enc.fit(train[["ville", "paiement"]])
print(list(enc.get_feature_names_out()))
# ['ville_Bamako', 'ville_Kayes', 'ville_Sikasso',
# 'paiement_Especes', 'paiement_Orange Money', 'paiement_Wave']
print(enc.transform(train[["ville", "paiement"]]).shape) # (8, 6)
print(enc.transform(test[["ville", "paiement"]]).shape) # (3, 6)
```

À retenir

- Séparer entraînement et test AVANT de calculer médianes, moyennes ou encodeurs : sinon, fuite de données.
- La médiane remplace mieux les valeurs manquantes numériques que la moyenne, car elle résiste aux valeurs extrêmes.
- L'encodage one-hot crée une colonne 0/1 par catégorie ; handle_unknown='ignore' évite le plantage sur une catégorie nouvelle.

Exercice

Une nouvelle commande arrive en production : ville = « Ségou », paiement = « Wave ». En utilisant l'encodeur de l'exemple (ajusté sur Bamako, Kayes, Sikasso), écris le vecteur encodé complet de cette commande sur les six colonnes produites, et explique en deux phrases pourquoi le programme ne plante pas et ce que ce vecteur « dit » au modèle.

✔ Corrigé
Vérifié par exécution : l'encodage de « Ségou » donne [0, 0, 0] pour les trois colonnes de ville, et « Wave » donne [0, 0, 1] sur les colonnes de paiement ; le vecteur complet est donc [0, 0, 0, 0, 0, 1]. Le programme ne plante pas grâce à handle_unknown='ignore' : la catégorie inconnue est traitée comme « aucune des villes connues ». Pour le modèle, cette commande ressemble à une commande sans ville identifiée mais payée par Wave : la prédiction reste possible, mais on a perdu l'information géographique — si Ségou devient fréquente, il faudra réentraîner l'encodeur (et le modèle) en l'incluant.

Leçon 2 — Arbres de décision et forêts aléatoires

Un arbre de décision prédit en posant des questions en cascade : « l'assiduité dépasse-t-elle 73,5 % ? » puis, selon la réponse, une autre question, jusqu'à une feuille qui annonce la classe. Notre jeu de travail, construit pour ce cours, décrit 20 étudiants par leurs heures de révision hebdomadaires, leur assiduité et leur nombre d'exercices faits, avec le résultat : admis (9 étudiants) ou non admis (11). Il contient volontairement deux cas « injustes » — un étudiant assidu et travailleur non admis, un autre peu assidu mais admis — parce que les données réelles en contiennent toujours. Séparé en 15 lignes d'entraînement et 5 de test, il donne des résultats modestes et honnêtes, parfaits pour apprendre.

L'arbre limité à une profondeur de 2 obtient 0,933 d'exactitude en entraînement et 0,80 en test. Sa règle tient en une phrase : assiduité ≤ 73,5 % → non admis, sinon admis. Autorisé jusqu'à la profondeur 3, le même arbre mémorise l'entraînement (1,00) mais tombe à 0,60 en test : c'est le surapprentissage, photographié en chiffres. Plus un arbre est profond, plus il épouse les accidents des données au lieu d'en apprendre la structure. Limiter la profondeur, exiger un nombre minimal d'exemples par feuille, ou moyenner beaucoup d'arbres sont les trois remèdes classiques.

La forêt aléatoire entraîne 100 arbres sur des échantillons et des sous-ensembles de variables tirés au hasard, puis fait voter l'ensemble. Ici, elle obtient 1,00 en entraînement et 0,60 en test — moins bien que le petit arbre sur ce test précis. Faut-il conclure que l'arbre est meilleur ? Non : le test ne compte que 5 étudiants, et la différence entre 0,80 et 0,60 vaut exactement un étudiant. C'est la leçon la plus importante de cette leçon : sur de petits jeux, un seul score ne prouve rien ; il faudra la validation croisée du TD 2 pour comparer sérieusement. La forêt fournit au passage les importances des variables : assiduité 0,449, exercices 0,374, heures 0,177 — l'assiduité « explique » le plus, sans que cela prouve un lien de cause à effet.

L'essentiel
Un arbre trop profond mémorise (train 1,00, test 0,60) ; un arbre limité généralise mieux (0,80 en test). Une forêt vote entre 100 arbres et donne les importances des variables. Sur 5 lignes de test, un écart de score peut ne valoir qu'un seul exemple : ne jamais conclure trop vite.

Exemple — Arbre contre forêt sur la réussite des étudiants (résultats réels)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Jeu construit pour le cours : 20 etudiants
donnees = [
[2,55,5,0],[3,60,8,0],[5,70,12,0],[4,65,10,0],
[6,75,15,1],[7,80,18,1],[8,85,20,1],[5,90,22,1],
[3,85,14,1],[9,95,25,1],[1,50,3,0],[6,60,9,0],
[10,92,28,1],[4,78,16,1],[7,68,11,0],[2,62,7,0],
[6,88,19,0],[5,58,7,1],[8,72,13,0],[3,66,10,0],
]
cols = ["heures", "assiduite", "exercices", "admis"]
df = pd.DataFrame(donnees, columns=cols)
X, y = df[cols[:3]], df["admis"]
Xtr, Xte, ytr, yte = train_test_split(
X, y, test_size=0.25, random_state=42) # 15 train / 5 test

arbre = DecisionTreeClassifier(max_depth=2, random_state=42)
arbre.fit(Xtr, ytr)
print("arbre prof.2 : train", round(arbre.score(Xtr, ytr), 3),
"| test", round(arbre.score(Xte, yte), 3)) # 0.933 | 0.8
arbre3 = DecisionTreeClassifier(max_depth=3, random_state=42)
arbre3.fit(Xtr, ytr)
print("arbre prof.3 : train", round(arbre3.score(Xtr, ytr), 3),
"| test", round(arbre3.score(Xte, yte), 3)) # 1.0 | 0.6
foret = RandomForestClassifier(n_estimators=100, random_state=42)
foret.fit(Xtr, ytr)
print("foret : train", round(foret.score(Xtr, ytr), 3),
"| test", round(foret.score(Xte, yte), 3)) # 1.0 | 0.6
print("importances :", dict(zip(cols[:3],
foret.feature_importances_.round(3))))
# {'heures': 0.177, 'assiduite': 0.449, 'exercices': 0.374}
```

À retenir

- Profondeur = bouton de réglage du surapprentissage : profondeur 3 → train 1,00 / test 0,60 ; profondeur 2 → 0,933 / 0,80.
- La forêt aléatoire moyenne 100 arbres entraînés sur des tirages différents : plus stable, mais pas magique sur de très petits jeux.
- Les importances de variables décrivent ce que le modèle utilise, pas des causes : l'assiduité (0,449) domine ici, devant les exercices (0,374) et les heures (0,177).

Exercice

À l'aide de l'arbre de profondeur 2 de l'exemple (règle : assiduité ≤ 73,5 % → non admis ; sinon admis), prédis à la main le résultat de deux nouveaux étudiants : Awa (6 heures, 80 % d'assiduité, 10 exercices) et Modibo (9 heures, 70 % d'assiduité, 25 exercices). Explique ensuite pourquoi l'arbre ignore totalement les heures de Modibo, et ce que cela révèle sur les limites d'un modèle très simple.

✔ Corrigé
Awa : assiduité 80 % > 73,5 % → l'arbre prédit « admis ». Modibo : assiduité 70 % ≤ 73,5 % → l'arbre prédit « non admis », malgré ses 9 heures et ses 25 exercices. L'arbre de profondeur 2 n'a retenu que l'assiduité comme question utile ; toutes les autres variables sont ignorées dans ses décisions, même quand elles contredisent le verdict. C'est le prix d'un modèle très simple : lisible et robuste, mais aveugle aux nuances. La forêt, elle, utilise les trois variables (importances 0,449 / 0,374 / 0,177) — d'où l'intérêt de comparer plusieurs modèles plutôt que de n'en croire qu'un.

Leçon 3 — Réduction de dimension et sélection de variables

Trop de variables nuit : calculs lents, modèles qui se perdent dans le bruit, explications impossibles. Deux familles de remèdes existent. La réduction de dimension fabrique de nouvelles variables, combinaisons des anciennes : c'est l'analyse en composantes principales (ACP, ou PCA en anglais). La sélection de variables, au contraire, garde un sous-ensemble des variables d'origine, inchangées : on perd moins en interprétation, mais on renonce aux combinaisons astucieuses. Les deux approches se complètent et obéissent à la même discipline : toutes leurs statistiques (moyennes, écarts-types, scores) s'apprennent sur l'entraînement seul, sous peine — encore — de fuite de données.

Notre exemple de PCA porte sur 12 personnes décrites par quatre mesures corporelles fortement corrélées : taille, poids, pointure et tour de poitrine. Après centrage-réduction obligatoire (StandardScaler : sans lui, la taille en centimètres écraserait la pointure dans les calculs), la première composante capte à elle seule 96,0 % de la variance, la deuxième 3,7 %, les deux dernières presque rien (0,2 % et 0,0 %). Les contributions à la première composante sont presque égales (0,50 ; 0,49 ; 0,50 ; 0,51) : cette composante est tout simplement « la taille générale » du corps. La deuxième oppose le poids (+0,65) à la taille (−0,55) : une forme de « corpulence à taille égale ». Quatre mesures se résument donc honnêtement à deux nombres sans perdre que 0,3 % de l'information.

Pour la sélection, reprenons les étudiants de la leçon 2 et demandons au test F (f_classif) quelles variables distinguent le mieux admis et non admis : exercices F = 13,19 (p ≈ 0,0019), assiduité F = 12,78 (p ≈ 0,0022), heures F = 3,93 (p ≈ 0,0628). SelectKBest avec k = 2 retient donc assiduité et exercices — le même duo de tête que les importances de la forêt, ce qui renforce la confiance. Mais attention à l'interprétation : ces scores mesurent une séparation statistique dans CE jeu de données, pas un mécanisme causal. Et répétons la règle d'or : si la sélection est faite sur tout le jeu avant la séparation, les variables choisies ont déjà « vu » les réponses du test — l'exercice supplémentaire 3 montrera à quel point cette fuite peut gonfler les scores, jusqu'à 0,77 d'exactitude sur du bruit pur.

L'essentiel
PCA transforme les variables en composantes indépendantes classées par variance expliquée (ici 96,0 % puis 3,7 %) ; SelectKBest garde les variables d'origine les mieux notées (assiduité et exercices). Dans les deux cas : centrage-réduction et apprentissage sur l'entraînement seul.

Exemple — ACP sur des mesures corporelles corrélées (résultats réels)

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 12 personnes, mesures correlees (jeu construit pour le cours)
donnees = [
[165, 60, 40, 78], [172, 68, 42, 84], [158, 52, 38, 72],
[180, 78, 44, 92], [168, 63, 41, 80], [175, 72, 43, 88],
[160, 55, 39, 74], [185, 85, 45, 96], [170, 80, 42, 90],
[162, 58, 40, 76], [176, 66, 43, 84], [169, 64, 41, 81],
]
cols = ["taille_cm", "poids_kg", "pointure", "tour_poitrine_cm"]
df = pd.DataFrame(donnees, columns=cols)

Z = StandardScaler().fit_transform(df) # centrage-reduction
pca = PCA().fit(Z)
print("variance expliquee :",
pca.explained_variance_ratio_.round(3))
# [0.96 0.037 0.002 0. ]
print("cumulee PC1+PC2 :",
round(pca.explained_variance_ratio_[:2].sum(), 3)) # 0.997
print("contributions PC1 :",
dict(zip(cols, pca.components_[0].round(2))))
# taille 0.50, poids 0.49, pointure 0.50, poitrine 0.51
print("contributions PC2 :",
dict(zip(cols, pca.components_[1].round(2))))
# taille -0.55, poids 0.65, pointure -0.41, poitrine 0.32
```

À retenir

- Sans centrage-réduction, la PCA est dominée par les variables aux grandes unités : toujours StandardScaler avant.
- Ici, 2 composantes sur 4 conservent 99,7 % de la variance : les quatre mesures corporelles ne portaient en réalité que deux informations.
- Sélection et réduction se calculent sur l'entraînement seul ; sélectionner sur tout le jeu est une fuite de données (voir l'exercice supplémentaire 3).

Exercice

À partir des parts de variance de l'exemple (PC1 : 0,960 ; PC2 : 0,037 ; PC3 : 0,002 ; PC4 : 0,000), réponds : (1) combien de composantes faut-il pour conserver au moins 95 % de l'information ? (2) combien pour au moins 99,5 % ? (3) Un collègue lance la PCA directement sur les données brutes, sans StandardScaler : quelle variable risque

de dominer artificiellement la première composante, et pourquoi ?

✔ Corrigé
(1) Une seule composante suffit : PC1 conserve déjà 96,0 % de variance, au-dessus du seuil de 95 %. (2) Il en faut deux : PC1 + PC2 = 0,960 + 0,037 = 0,997, soit 99,7 % ≥ 99,5 % (avec PC1 seule on reste à 96,0 %). (3) La taille en centimètres (valeurs autour de 158 à 185) possède une variance numérique bien plus grande que la pointure (38 à 45) ou le tour de poitrine : sans centrage-réduction, la PCA confond « grande échelle de mesure » et « information importante », et la première composante ne refléterait presque que la taille brute. Le StandardScaler met chaque variable à moyenne 0 et écart-type 1 pour comparer leurs rôles à armes égales.

Leçon 4 — Séries temporelles : prévisions simples

Une série temporelle est une suite d'observations datées : ventes mensuelles, températures quotidiennes, nombre de passagers. Trois ingrédients la composent souvent : une tendance (la direction générale), une saisonnalité (un motif qui se répète, par exemple le pic des ventes de décembre) et du bruit. La règle qui distingue radicalement ce domaine du reste du machine learning : le temps ne se mélange pas. Valider un modèle de prévision en tirant le test au hasard reviendrait à s'entraîner sur le futur pour « prédire » le passé : les scores seraient flatteurs et inutilisables. L'évaluation honnête est chronologique : on s'entraîne sur le passé, on prévoit la période suivante, on compare au réel — exactement comme en situation réelle.

Trois méthodes de référence, volontairement simples, servent d'étalons. La prévision naïve répète la dernière valeur connue. La moyenne mobile d'ordre 3 moyenne les trois dernières valeurs : elle lisse le bruit mais réagit en retard aux changements. La saisonnière naïve répète la valeur d'il y a douze mois : elle parie que le calendrier gouverne la série. Sur les ventes mensuelles de notre boutique (24 mois, de janvier 2023 à décembre 2024, données construites pour le cours), l'évaluation chronologique sur les six derniers mois donne des erreurs moyennes absolues (MAE) parlantes : saisonnière naïve 15,17 ; naïve 15,5 ; moyenne mobile 22,89 — en milliers de FCFA.

La saisonnière naïve gagne d'un cheveu, et cela s'explique : la boutique a un vrai rythme annuel — chaque mois de 2024 dépasse son jumeau de 2023 d'un montant assez régulier (13 à 14 milliers de FCFA, 25 pour décembre) — donc recopier l'année précédente est presque juste, à la croissance près. La naïve suit de très près (15,5) car la série avance sans rupture d'un mois à l'autre. La moyenne mobile, elle, est nettement battue (22,89) : sur une série en croissance, moyenner les trois derniers mois revient à viser systématiquement trop bas, et le pic de fin d'année aggrave encore son retard (son erreur atteint 51,7 en décembre). Pour janvier 2025, les trois prévisions divergent : naïve 235 (elle recopie le pic de décembre — peu crédible pour janvier, mois creux), moyenne mobile 205,7 (encore gonflée par novembre et décembre), saisonnière 120 (le niveau de janvier 2024… mais sans la croissance de l'année, donc sans doute trop basse). Le métier commence ici : présenter la fourchette, justifier le choix — par exemple une saisonnière relevée de la croissance observée — et savoir que toute méthode avancée (lissage exponentiel, ARIMA) devra d'abord battre ces modestes références pour mériter son nom.

L'essentiel
En séries temporelles, la validation est toujours chronologique : jamais de découpage aléatoire. Trois références à toujours calculer : naïve (dernière valeur), moyenne mobile (lissage, mais retard), saisonnière naïve (même période l'an passé). Ici, MAE respectives : 15,5 ; 22,89 ; 15,17 — la saisonnière gagne d'un cheveu, la moyenne mobile est nettement battue.

Exemple — Prévoir les ventes d'une boutique : trois méthodes, une seule vérité chronologique

```
import numpy as np

# Ventes mensuelles (milliers de FCFA), janv. 2023 -> dec. 2024
ventes = [110, 105, 120, 130, 125, 140, 150, 145, 155, 170, 185, 210,
120, 118, 132, 142, 138, 152, 163, 158, 168, 183, 199, 235]
train, test = ventes[:18], ventes[18:] # test = 6 derniers mois

def mae(vrais, prevus):
return np.mean([abs(v - p) for v, p in zip(vrais, prevus)])

p_naive, p_sais, p_mm = [], [], []
for i in range(len(test)):
t = len(train) + i
p_naive.append(ventes[t - 1]) # derniere valeur
p_sais.append(ventes[t - 12]) # il y a 12 mois
p_mm.append(np.mean(ventes[t - 3:t])) # moyenne mobile 3
print("MAE naive :", round(mae(test, p_naive), 2)) # 15.5
print("MAE saisonniere :", round(mae(test, p_sais), 2)) # 15.17
print("MAE moyenne mobile :", round(mae(test, p_mm), 2)) # 22.89
# Previsions pour janvier 2025
print("janv. 2025, naive :", ventes[-1]) # 235
print("janv. 2025, saisonniere :", ventes[-12]) # 120
print("janv. 2025, moyenne mob.:", round(np.mean(ventes[-3:]), 1)) # 205.7
```

À retenir

- Ne jamais mélanger une série temporelle au hasard : l'entraînement précède toujours le test dans le temps.
- La moyenne mobile lisse mais réagit en retard ; après un pic, elle reste fausse pendant plusieurs périodes.
- Toute méthode avancée doit battre la prévision naïve sur une évaluation chronologique, sinon elle n'apporte rien.

Exercice

Une vendeuse de jus suit ses ventes hebdomadaires (en milliers de FCFA) : [20, 22, 21, 24]. (1) Calcule la prévision de la semaine 5 par la méthode naïve, puis par la moyenne mobile d'ordre 3. (2) Les quatre semaines sont mélangées au hasard avant l'évaluation : explique en trois phrases pourquoi le résultat obtenu ne vaudrait rien.

✔ Corrigé
(1) Prévision naïve : on répète la dernière valeur connue, soit 24. Moyenne mobile d'ordre 3 : on moyenne les trois dernières semaines, (22 + 21 + 24) / 3 = 67 / 3 ≈ 22,33. La naïve parie sur la poursuite du niveau le plus récent ; la moyenne mobile lisse et répond plus bas. (2) Mélanger la série détruit l'ordre temporel : le modèle « apprendrait » sur des semaines postérieures à celle qu'il doit prévoir, c'est-à-dire sur le futur. L'erreur mesurée serait artificiellement faible, car en réalité on ne connaît jamais l'avenir au moment de prévoir. L'évaluation chronologique (entraîner sur le passé, tester sur la suite) est la seule qui reproduise les conditions réelles d'usage.

Leçon 5 — Systèmes de recommandation : initiation

Chaque jour, des systèmes choisissent ce qu'on te propose : un produit à acheter, une vidéo à regarder, un article à lire. La technique fondatrice, le filtrage collaboratif, repose sur une intuition simple : les personnes qui ont aimé les mêmes choses hier aimeront probablement les mêmes choses demain. Elle travaille sur une matrice utilisateurs × articles remplie de notes — et surtout de trous : personne n'a tout noté. Notre exemple, construit pour le cours, croise 5 clientes et clients d'une boutique avec 6 produits notés de 1 à 5 ; le zéro signifie « pas encore noté », et c'est précisément ces zéros que le système cherche à remplir intelligemment.

Pour décider qui se ressemble, on compare les vecteurs de notes par leur similarité cosinus : deux utilisateurs sont d'autant plus proches que leurs vecteurs « pointent » dans la même direction. Les résultats réels sur notre matrice : Ibrahim ressemble à Awa à 0,991 (presque des jumeaux de goût), Moussa à 0,888, Fatou à 0,570 et Mariam à 0,494 seulement. La note prédite pour un article non noté est alors la moyenne des notes des voisins, pondérée par ces similarités : les voisins proches pèsent plus lourd. Pour le Sucre, qu'Awa n'a jamais noté : Moussa a mis 3, Fatou 4, Mariam 5 (Ibrahim, lui, ne l'a pas noté non plus et ne peut donc pas voter) ; la moyenne pondérée donne 3,80 sur 5.

Une prédiction de 3,80 dépasse le seuil de recommandation qu'on se fixe souvent (par exemple 3,5) : la boutique proposera donc le Sucre à Awa, par exemple dans un message « les clientes qui vous ressemblent ont aussi acheté… ». Il faut néanmoins connaître les limites du mécanisme : un nouvel utilisateur sans aucune note est invisible (problème du démarrage à froid), un article jamais noté ne peut être recommandé, et les articles populaires écrasent les autres dans les suggestions. Les plateformes réelles combinent donc le filtrage collaboratif avec le contenu des articles et des garde-fous de diversité — mais le cœur du raisonnement reste celui de cette leçon.

L'essentiel
Filtrage collaboratif : similarité cosinus entre utilisateurs, puis note prédite = moyenne des notes des voisins pondérée par la similarité. Ici, Awa–Ibrahim : 0,991 ; note prédite d'Awa pour le Sucre : 3,80/5 → on recommande.

Exemple — Recommander un produit par similarité cosinus (résultats réels)

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

produits = ["Riz", "Huile", "Sucre", "Thé", "Café", "Savon"]
utilisateurs = ["Awa", "Moussa", "Fatou", "Ibrahim", "Mariam"]
# Notes de 1 a 5 ; 0 = pas encore note
notes = np.array([
[5, 3, 0, 4, 5, 2], # Awa
[4, 2, 3, 5, 4, 0], # Moussa
[1, 5, 4, 2, 1, 5], # Fatou
[5, 4, 0, 4, 5, 3], # Ibrahim
[2, 5, 5, 1, 0, 4], # Mariam
], dtype=float)

sim = cosine_similarity(notes)
np.fill_diagonal(sim, 0) # on ignore la similarite avec soi-meme
for u, s in zip(utilisateurs, sim[0]):
print("similarite Awa -", u, ":", round(s, 3))
# Ibrahim 0.991 | Moussa 0.888 | Fatou 0.570 | Mariam 0.494

# Note predite d'Awa pour le Sucre (colonne 2, non note par Awa)
j = 2
num, den = 0.0, 0.0
for u in range(len(utilisateurs)):
if u != 0 and notes[u, j] > 0:
num += sim[0, u] * notes[u, j]
den += abs(sim[0, u])
print("note predite Awa / Sucre :", round(num / den, 2)) # 3.8
```

À retenir

- La similarité cosinus vaut 1 pour des goûts identiques en direction, proche de 0 pour des goûts sans rapport.
- Seuls les voisins ayant réellement noté l'article votent pour sa note prédite, avec un poids égal à leur similarité.
- Démarrage à froid et matrice creuse sont les limites natives du filtrage collaboratif : il faut des notes pour recommander.

Exercice

Mariam n'a jamais noté le Café (colonne 4). Ses similarités avec les autres sont : Awa 0,494 ; Moussa 0,539 ; Fatou 0,965 ; Ibrahim 0,572. Les notes du Café : Awa 5, Moussa 4, Fatou 1, Ibrahim 5. Calcule la note prédite de Mariam pour le Café, puis explique pourquoi elle est nettement plus basse que celle d'Awa pour le Sucre (3,80) alors que tous les ingrédients semblent favorables.

✔ Corrigé
Tous les voisins de Mariam ont noté le Café, donc tous votent. Numérateur : 0,494×5 + 0,539×4 + 0,965×1 + 0,572×5 = 2,470 + 2,156 + 0,965 + 2,860 = 8,451. Dénominateur : 0,494 + 0,539 + 0,965 + 0,572 = 2,570. Note prédite : 8,451 / 2,570 ≈ 3,29 (valeur confirmée par exécution). L'explication tient en un nom : Fatou. C'est la voisine la plus semblable à Mariam (0,965, un poids énorme), et elle a détesté le Café (note 1). Sa voix pèse plus que celles d'Awa, Moussa et Ibrahim réunies en proportion, et tire la prédiction vers le bas. C'est exactement le comportement voulu par le filtrage collaboratif : faire confiance d'abord aux goûts les plus proches.

Leçon 6 — Déployer un modèle : API et suivi des performances

Un modèle qui dort dans un notebook ne sert à personne. Le déployer, c'est le rendre appelable par d'autres programmes : on sauvegarde le modèle entraîné (par exemple avec joblib), puis on l'enrobe dans une petite API web — FastAPI ou Flask en Python. Concrètement, un point d'accès « POST /predire » reçoit un JSON du type {heures : 6, assiduite : 80, exercices : 15}, vérifie que les champs existent et sont dans des bornes sensées, appelle le modèle et renvoie sa réponse {admis_predit : 1, probabilite : …}. Ce découplage est précieux : le site web, l'application mobile et le modèle évoluent indépendamment, et le même modèle peut servir plusieurs produits.

Le jour du lancement n'est pas la fin du travail, mais le début de la surveillance. Un modèle se dégrade parce que le monde change : dérive des données quand les entrées changent de profil (nouvelle population, nouvelle saison), dérive conceptuelle quand, à entrées égales, la bonne réponse change (barème d'examen durci, comportement des clients modifié). C'est pourquoi un déploiement sérieux journalise chaque prédiction avec ses entrées, puis compare régulièrement les nouvelles distributions à celles de l'entraînement et — dès que les vraies réponses sont connues — les performances réelles à celles du lancement.

Notre suivi chiffré, exécuté sur le modèle d'admission réentraîné sur les 20 étudiants, le montre de façon frappante. Trois mois plus tard, le lot de production (20 nouveaux étudiants) donne une exactitude de 0,80 : en apparence, tout va bien. Mais la précision s'effondre à 0,556 : le modèle a prédit 9 admis alors qu'il n'y en a eu que 5. Les entrées, elles, ont à peine bougé (heures moyennes 5,2 → 4,9 ; assiduité 72,7 → 71,4), tandis que le taux d'admis réel est passé de 0,45 à 0,25 : c'est la relation entre le profil et l'admission qui a changé — une dérive conceptuelle, invisible si l'on ne surveille que l'exactitude globale. La bonne réaction : alerter, constituer un jeu étiqueté récent et réentraîner ; la mauvaise : se féliciter du 0,80 et ne rien faire.

L'essentiel
Déployer = sauvegarder le modèle, l'exposer par une API validée et journalisée. Surveiller = comparer entrées et performances à une référence. Ici, l'exactitude semble stable (0,80) mais la précision chute à 0,556 : taux d'admis réel passé de 0,45 à 0,25, c'est une dérive conceptuelle qu'un seul indicateur aurait cachée.

Exemple — Suivi d'un modèle en production : la dérive en chiffres (résultats réels)

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score

donnees = [
[2,55,5,0],[3,60,8,0],[5,70,12,0],[4,65,10,0],
[6,75,15,1],[7,80,18,1],[8,85,20,1],[5,90,22,1],
[3,85,14,1],[9,95,25,1],[1,50,3,0],[6,60,9,0],
[10,92,28,1],[4,78,16,1],[7,68,11,0],[2,62,7,0],
[6,88,19,0],[5,58,7,1],[8,72,13,0],[3,66,10,0],
]
cols = ["heures", "assiduite", "exercices", "admis"]
df = pd.DataFrame(donnees, columns=cols)
X, y = df[cols[:3]], df["admis"]
# Modele final : reentraine sur toutes les donnees connues
modele = RandomForestClassifier(n_estimators=100, random_state=42)
modele.fit(X, y)
print("taux d'admis a l'entrainement :", round(y.mean(), 2)) # 0.45

# Lot de production, 3 mois plus tard (nouvelle promotion)
prod = [
[1,58,4,0],[2,64,6,0],[3,58,9,0],[4,72,10,0],
[5,66,12,0],[6,70,13,0],[5,74,14,1],[7,76,15,1],
[8,82,19,1],[3,70,8,0],[9,88,22,1],[2,52,5,0],
[6,78,16,0],[4,62,9,0],[7,84,17,0],[5,80,15,0],
[8,90,21,1],[3,60,7,0],[6,68,11,0],[4,76,13,0],
]
dp = pd.DataFrame(prod, columns=cols)
pred = modele.predict(dp[cols[:3]])
print("exactitude production :", round(
modele.score(dp[cols[:3]], dp["admis"]), 3)) # 0.8
print("precision production :", round(
precision_score(dp["admis"], pred, zero_division=0), 3)) # 0.556
print("admis reels :", int(dp["admis"].sum()),
"| admis predits :", int(pred.sum())) # 5 | 9
```

À retenir

- Une API de prédiction valide ses entrées, journalise ses sorties et renvoie une réponse structurée (prédiction, éventuellement probabilité).
- Surveiller plusieurs indicateurs : l'exactitude seule peut sembler stable pendant que la précision s'effondre (0,556 ici).
- Entrées stables + réponses différentes = dérive conceptuelle ; la réponse adaptée est un réentraînement sur des données récentes étiquetées.

Exercice

Sur un nouveau lot de production, un modèle prédit 12 admissions ; 7 de ces prédictions sont correctes, et le nombre réel d'admis est de 8. (1) Calcule la précision et le rappel du modèle sur ce lot. (2) En deux phrases, explique ce que chacun des deux nombres raconte et pourquoi un responsable d'admissions devrait regarder le

premier en priorité.

✔ Corrigé
(1) Précision = vrais positifs / prédits positifs = 7 / 12 ≈ 0,583. Rappel = vrais positifs / positifs réels = 7 / 8 = 0,875. (2) Le rappel dit que le modèle retrouve presque tous les étudiants qui seront admis (7 sur 8) : peu d'admis lui échappent. La précision dit que, parmi les 12 dossiers qu'il annonce admis, 5 ne le seront pas : si l'établissement engage des moyens (places, bourses, encadrement) sur la foi de ces prédictions, il gaspille près de la moitié de l'effort. C'est donc la précision qui doit alerter en priorité ici — exactement le signal que l'exemple de la leçon faisait apparaître (0,556) derrière une exactitude rassurante.

Leçon 7 — Projet : prédiction à partir de données réelles

Ce projet assemble tout le cours en une démarche complète, sur un vrai jeu de données : « Diabetes », intégré à scikit-learn (fonction load_diabetes). Il provient d'une étude publiée (Efron, Hastie, Johnstone et Tibshirani, 2004) et décrit 442 patients par 10 variables — âge, sexe, indice de masse corporelle, tension artérielle et six mesures sanguines (s1 à s6) — avec pour cible un score de progression de la maladie un an plus tard (de 25 à 346, moyenne 152,1). La question métier : peut-on estimer cette progression dès l'examen initial, pour prioriser le suivi des patients ? Le jeu est réel et public, cité honnêtement ; ses variables sanguines sont anonymisées, ce qui est aussi une leçon : travailler sur des données de santé impose ce genre de précautions.

La démarche suit l'ordre du cours. D'abord, une référence naïve : prédire pour chaque patient la moyenne des progressions du jeu d'entraînement. Sur le test (séparation aléatoire 80/20 — légitime ici, car les patients sont indépendants, ce n'est pas une série temporelle), elle obtient une erreur moyenne absolue (MAE) de 64,0 et une RMSE de 73,2. Ensuite, une forêt aléatoire de 100 arbres sur les mêmes données : MAE 44,1, RMSE 54,3, R^2 = 0,443. L'amélioration est réelle — environ 31 % d'erreur moyenne en moins que la référence — mais le R^2 rappelle que plus de la moitié de la variabilité reste inexpliquée : le modèle aide à trier, il ne « connaît » pas l'avenir d'un patient.

Les importances de la forêt ouvrent la discussion plutôt qu'elles ne la ferment : l'indice de masse corporelle domine (0,355), suivi de la mesure s5 (0,231) et de la tension (0,088) ; le sexe ne compte presque pas (0,010). Un tel résultat sert deux fois : au médecin, qui y voit un facteur de vigilance plausible ; et à l'équipe technique, qui peut envisager un modèle plus simple fondé sur quelques variables. Le bilan du projet doit enfin lister ses limites sans les maquiller : données datées et issues d'une population précise, variables anonymisées dont on ne connaît pas le sens médical exact, absence de validation sur un hôpital différent. Un projet de data mining se juge autant à l'honnêteté de ce bilan qu'à ses scores.

L'essentiel
Projet complet sur le jeu réel Diabetes (442 patients) : référence naïve MAE 64,0 ; forêt aléatoire MAE 44,1, RMSE 54,3, R^2 0,443. Variables clés : IMC (0,355) et s5 (0,231). Un bon projet compare à une référence, quantifie, interprète — et déclare ses limites.

Exemple — Prédire la progression du diabète avec une forêt aléatoire (résultats réels)

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

X, y = load_diabetes(return_X_y=True, as_frame=True)
print("patients :", X.shape[0], "| variables :", X.shape[1]) # 442 | 10
Xtr, Xte, ytr, yte = train_test_split(
X, y, test_size=0.2, random_state=42)

# Reference naive : toujours predire la moyenne de l'entrainement
ref = np.full_like(yte, ytr.mean(), dtype=float)
print("reference : MAE =", round(mean_absolute_error(yte, ref), 1),
"| RMSE =", round(np.sqrt(mean_squared_error(yte, ref)), 1))
# reference : MAE = 64.0 | RMSE = 73.2

modele = RandomForestRegressor(n_estimators=100, random_state=42)
modele.fit(Xtr, ytr)
pred = modele.predict(Xte)
print("foret : MAE =", round(mean_absolute_error(yte, pred), 1),
"| RMSE =", round(np.sqrt(mean_squared_error(yte, pred)), 1),
"| R2 =", round(modele.score(Xte, yte), 3))
# foret : MAE = 44.1 | RMSE = 54.3 | R2 = 0.443
imp = sorted(zip(X.columns, modele.feature_importances_),
key=lambda t: t[1], reverse=True)
print("top 3 importances :", [(n, round(v, 3)) for n, v in imp[:3]])
# [('bmi', 0.355), ('s5', 0.231), ('bp', 0.088)]
```

À retenir

- Tout projet commence par une référence naïve chiffrée : sans elle, impossible de dire ce que le modèle apporte (ici, MAE de 64,0 → 44,1).
- MAE et RMSE ne racontent pas la même chose : la RMSE, qui pénalise les grosses erreurs au carré, est toujours supérieure ou égale à la MAE.
- Présenter un modèle, c'est aussi présenter ses limites : population des données, variables anonymisées, absence de validation externe.

Exercice

Pour quatre patients du test, les progressions réelles sont [120, 200, 90, 310]. Un modèle simpliste prévoit 150 pour tout le monde. (1) Calcule la MAE de ce modèle sur ces quatre patients. (2) Compare-la à la MAE de la forêt (44,1) et à celle de la référence moyenne du projet (64,0), et dis en une phrase si ce modèle simpliste serait

acceptable.

✔ Corrigé
(1) Erreurs absolues : |120 − 150| = 30 ; |200 − 150| = 50 ; |90 − 150| = 60 ; |310 − 150| = 160. Somme : 300 ; MAE = 300 / 4 = 75,0. La grosse erreur sur le quatrième patient (160) pèse lourd : avec la RMSE, elle pèserait encore plus, car 160^2 dominerait le calcul. (2) Avec 75,0, le modèle simpliste fait pire que la simple moyenne du projet (64,0) et très loin de la forêt (44,1) : il ne serait pas acceptable, car il n'apprend rien des caractéristiques des patients — il sert uniquement d'étalon pour mesurer ce que les vrais modèles apportent.

Travaux dirigés (TD)

TD 1 — Nettoyage d'une série de notes et chasse aux valeurs aberrantes

Un enseignant saisit les notes de sa classe dans pandas : [12, 15, None, 9, 200, 14, 11, 13, 10, 16] (le None représente une copie non rendue). Travail demandé : (1) compte les valeurs manquantes ; (2) calcule la médiane des notes renseignées ; (3) calcule Q1, Q3, l'écart interquartile (IQR) et la borne haute Q3 + 1,5 × IQR ; (4) identifie la ou les valeurs aberrantes et produis la série nettoyée ; (5) compare la moyenne avant et après nettoyage, et explique en trois phrases pourquoi la médiane bouge si peu alors que la moyenne s'effondre. Tous les nombres doivent être calculés, pas estimés.

✔ Corrigé du TD 1
Résultats obtenus par exécution. (1) Il y a 1 valeur manquante. (2) Médiane des valeurs renseignées (triées : 9, 10, 11, 12, 13, 14, 15, 16, 200) : 13,0. (3) Q1 = 11,0, Q3 = 15,0, IQR = 4,0, borne haute = 15,0 + 1,5 × 4,0 = 21,0. (4) La note 200 dépasse la borne haute : c'est une valeur aberrante (une erreur de saisie quasi certaine, la note maximale possible étant 20). Série nettoyée, sans la valeur manquante ni l'aberrante : [12, 15, 9, 14, 11, 13, 10, 16]. (5) Moyenne avant nettoyage (avec 200, sans le manquant) : 33,33 ; après nettoyage : 12,5. La moyenne utilise chaque valeur dans son calcul : le 200 la tire violemment vers le haut. La médiane ne dépend que de l'ordre des valeurs : remplacer 200 par n'importe quel nombre supérieur à 21 laisserait la médiane à 13,0. C'est cette robustesse qui en fait la statistique de remplacement préférée pour les valeurs manquantes numériques.

TD 2 — Validation croisée : dépasser le verdict d'un seul découpage

Dans la leçon 2, la forêt aléatoire obtenait 0,60 d'exactitude sur un unique découpage entraînement/test du jeu des 20 étudiants, et le petit arbre 0,80. Un étudiant conclut : « l'arbre est meilleur que la forêt ». Travail demandé : (1) code une validation croisée à 4 plis (cross_val_score, cv = 4) de la forêt (100 arbres, random_state = 42) sur les 20 étudiants ; (2) donne les scores par pli, la moyenne et l'écart-type ; (3) rédige en cinq phrases maximum un avis argumenté sur la conclusion de l'étudiant, en mobilisant tes résultats.

✔ Corrigé du TD 2
Résultats obtenus par exécution : scores par pli [0,6 ; 0,8 ; 0,6 ; 0,8], exactitude moyenne 0,70, écart-type 0,10. Avis argumenté : la conclusion de l'étudiant n'est pas recevable. Son unique test comptait 5 étudiants : l'écart entre 0,60 et 0,80 y vaut exactement un étudiant, ce qui ne prouve rien. La validation croisée fait passer chaque étudiant par le rôle de test une fois : la forêt y vaut 0,70 en moyenne, entre ses deux scores extrêmes. L'écart-type de 0,10 rappelle en outre que, sur 20 lignes, toutes ces mesures restent fragiles. La bonne pratique est donc de comparer les modèles en validation croisée, avec les mêmes plis, et de ne trancher que si l'écart dépasse nettement la variabilité observée.

Travaux pratiques (TP)

TP 1 — Pipeline complet : préparer, entraîner et valider sans fuite

Objectif : assembler la chaîne professionnelle complète sur le jeu des 20 étudiants de la leçon 2, augmenté d'une variable catégorielle « filiere » (Info, Gestion ou Droit) attribuée à chaque étudiant. Étapes : (1) construis le DataFrame complet (heures, assiduite, exercices, filiere, admis) ; (2) crée un ColumnTransformer qui encode « filiere » en one-hot (handle_unknown = 'ignore') et laisse passer les trois variables numériques telles quelles ; (3) branche ce prétraitement et un arbre de décision (profondeur 2) dans un Pipeline unique ; (4) fais de même avec une forêt aléatoire de 100 arbres ; (5) évalue les deux pipelines en validation croisée à 4 plis, mêmes plis pour les deux ; (6) rédige trois phrases de conclusion : quel modèle retiendrais-tu, et pourquoi le Pipeline protège-t-il contre la fuite de données ? Livrable : le script et son compte rendu d'une demi-page.

✔ Attendus du TP 1
Résultats obtenus par exécution (validation croisée à 4 plis, random_state = 42). Arbre de profondeur 2 : scores par pli [0,6 ; 1,0 ; 0,8 ; 0,6], moyenne 0,75. Forêt aléatoire : scores par pli [0,6 ; 1,0 ; 0,6 ; 0,8], moyenne 0,75. Les deux modèles sont donc à égalité en moyenne, avec la même variabilité : sur un jeu de 20 lignes, on retiendra le modèle le plus simple à expliquer (l'arbre) ou la forêt si la stabilité prime, mais aucun écart ne permet de déclarer un vainqueur. Le Pipeline protège contre la fuite car le ColumnTransformer est réajusté à l'intérieur de chaque pli, uniquement sur les lignes d'entraînement du pli : les catégories et statistiques du pli de test ne participent jamais à sa préparation. C'est la manière industrielle de garantir ce que la leçon 1 faisait à la main : aucun apprentissage sur les données d'évaluation.

Exercices corrigés supplémentaires

Exercice 1

Calcule à la main la moyenne mobile d'ordre 2 de la série [10, 12, 11, 13, 15, 14] (ventes hebdomadaires, en milliers de FCFA). Donne la liste complète des valeurs de la moyenne mobile, puis les prévisions du mois 7 selon la méthode naïve et selon la

dernière valeur de ta moyenne mobile.

✔ Corrigé
Moyennes mobiles d'ordre 2 (moyenne de chaque paire de semaines consécutives) : (10+12)/2 = 11,0 ; (12+11)/2 = 11,5 ; (11+13)/2 = 12,0 ; (13+15)/2 = 14,0 ; (15+14)/2 = 14,5. La liste est donc [11,0 ; 11,5 ; 12,0 ; 14,0 ; 14,5] — cinq valeurs pour six semaines, la première semaine n'ayant pas de moyenne (il faut deux valeurs pour la calculer). Prévision naïve du mois 7 : la dernière valeur observée, 14. Prévision par la moyenne mobile : sa dernière valeur, 14,5. Les deux sont proches car la série termine sans rupture ; sur une série qui chute brutalement à la fin, la moyenne mobile, fidèle à son retard, annoncerait encore un chiffre trop optimiste.

Exercice 2

Deux clients sont représentés par leurs vecteurs de notes sur trois produits : A = [3, 1, 0] et B = [2, 2, 1]. Calcule leur similarité cosinus à la main (produit scalaire, normes), arrondie à trois décimales, puis vérifie ton résultat avec sklearn.metrics.pairwise.cosine_similarity. Conclus en une phrase : ces deux clients sont-ils proches ?

✔ Corrigé
Produit scalaire : 3×2 + 1×2 + 0×1 = 8. Norme de A : √(9 + 1 + 0) = √10 ≈ 3,162. Norme de B : √(4 + 4 + 1) = √9 = 3. Similarité cosinus : 8 / (3,162 × 3) = 8 / 9,487 ≈ 0,843. La vérification par scikit-learn donne exactement 0,843. Conclusion : une similarité de 0,843, proche de 1, indique des goûts nettement orientés dans le même sens — notamment un premier produit apprécié des deux (3 et 2) — donc un voisinage exploitable pour recommander à l'un ce que l'autre a aimé.

Exercice 3

Expérience de la fuite par sélection de variables. On fabrique 100 exemples décrits par 200 variables de bruit pur (loi normale), et une cible tirée au hasard : il n'existe donc AUCUN vrai lien à apprendre, et le hasard seul vaut environ 0,50 d'exactitude. Protocole A (fautif) : sélectionner les 10 « meilleures » variables avec SelectKBest sur tout le jeu, puis évaluer une régression logistique en validation croisée. Protocole B (correct) : placer la sélection DANS un Pipeline, pour qu'elle soit refaite dans chaque pli. Lance les deux protocoles, donne les deux exactitudes moyennes, et explique en quatre phrases d'où vient l'écart.

✔ Corrigé
Résultats obtenus par exécution (données de bruit, graine fixée) : protocole A, avec fuite : exactitude moyenne en validation croisée = 0,77. Protocole B, sans fuite : 0,48 — le niveau du hasard, comme attendu sur du bruit. Explication : parmi 200 variables sans rapport avec la cible, certaines, par pur hasard, « collent » bien aux réponses sur ces 100 exemples précis. Le protocole A choisit ces variables en regardant TOUT le jeu, y compris les lignes qui serviront ensuite de test : la sélection connaît déjà les réponses du test, et la validation croisée ne fait que mesurer cette triche. Dans le protocole B, la sélection est recalculée dans chaque pli sans voir le pli de test : les coïncidences disparaissent et le score retombe au hasard. Retenir : tout ce qui s'apprend sur les données (sélection, normalisation, encodage, statistiques d'imputation) doit vivre à l'intérieur de la validation, jamais avant elle.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quel est l'ordre correct pour préparer des données avant un modèle ?

   A) Calculer les médianes sur tout le jeu, puis séparer entraînement/test

   B) Séparer entraînement/test, puis calculer les médianes sur l'entraînement seul

   C) Peu importe, le résultat est identique

2. La fuite de données (data leakage), c'est…

   A) Un modèle qui perd ses fichiers

   B) Un jeu de données trop petit

   C) Une information du test ou du futur qui influence l'entraînement et fausse l'évaluation

3. Avec OneHotEncoder(handle_unknown='ignore'), une catégorie jamais vue à l'entraînement donne…

   A) Une erreur qui arrête le programme

   B) La colonne de la catégorie la plus fréquente

   C) Des zéros dans toutes les colonnes de cette variable

4. Un arbre obtient 1,00 d'exactitude en entraînement et 0,60 en test. Le diagnostic le plus probable est…

   A) Un excellent modèle

   B) Du surapprentissage : il a mémorisé l'entraînement

   C) Une fuite de données dans le test

5. Dans la leçon 2, quelle variable la forêt aléatoire juge-t-elle la plus importante ?

   A) Les heures de révision (0,177)

   B) Les exercices (0,374)

   C) L'assiduité (0,449)

6. Pourquoi centre-t-on et réduit-on les données avant une analyse en composantes principales ?

A) Pour aller plus vite

B) Pour que les variables aux grandes unités ne dominent pas artificiellement les composantes

C) Parce que la PCA exige des entiers

7. Pour évaluer une prévision de série temporelle, le découpage honnête est…

A) Un découpage chronologique : le passé entraîne, la suite sert de test

B) Un tirage aléatoire des mois entre entraînement et test

C) Le mois de décembre comme seul test

8. La moyenne mobile d'ordre 3, juste après un pic exceptionnel de ventes, va…

A) Ignorer totalement le pic

B) Rester gonflée par le pic pendant plusieurs périodes avant de redescendre

C) Devenir négative

9. Une similarité cosinus de 0,991 entre Awa et Ibrahim signifie que…

A) Ibrahim a mis de meilleures notes qu'Awa

B) Ils ont noté les mêmes produits, exactement pareil

C) Leurs goûts pointent presque exactement dans la même direction

10. En production, l'exactitude reste à 0,80 mais la précision tombe à 0,556. Il faut…

A) Se réjouir : l'exactitude est stable

B) Enquêter : c'est le signe d'une dérive, ici conceptuelle (taux d'admis réel passé de 0,45 à 0,25)

C) Changer la graine aléatoire et republier

Corrigés du QCM

1. Réponse B — Toute statistique apprise doit l'être sur l'entraînement seul : calculée sur tout le jeu, elle fait fuiter l'information du test dans l'entraînement (ici, la médiane des prix vaut 600 sur l'entraînement contre 525 sur tout le jeu).

2. Réponse C — La fuite gonfle les scores sans améliorer le modèle réel : l'exercice supplémentaire 3 montre 0,77 d'exactitude sur du bruit pur quand la sélection de variables fuit, contre 0,48 sans fuite.

3. Réponse C — Vérifié en exécution : « Ségou », inconnue de l'encodeur, est encodée [0, 0, 0] — la ligne reste exploitable, sans l'information de ville.

4. Réponse B — C'est le cas de l'arbre de profondeur 3 de la leçon 2 : parfait sur ce qu'il a vu, faible sur le nouveau. Limiter la profondeur (0,80 en test à profondeur 2) est un remède direct.

5. Réponse C — Importances mesurées : assiduité 0,449, exercices 0,374, heures 0,177. L'assiduité domine — sans que cela prouve qu'elle « cause » la réussite.

6. Réponse B — Sans StandardScaler, la taille en centimètres écraserait la pointure dans les calculs : la première composante refléterait l'échelle de mesure, pas l'information.

7. Réponse A — Mélanger le temps reviendrait à apprendre sur le futur pour prédire le passé : les scores seraient flatteurs et inutilisables. Le cours évalue sur les 6 derniers mois, dans l'ordre.

8. Réponse B — Le pic reste dans la fenêtre de calcul pendant 3 périodes : dans la leçon 4, l'erreur de la moyenne mobile atteint 51,7 (milliers de FCFA) en décembre, et sa prévision de janvier (205,7) reste gonflée par la fin d'année.

9. Réponse C — Le cosinus compare la direction des vecteurs de notes : 0,991, très proche de 1, indique des profils de goût quasi identiques — c'est ce qui rend leurs notes mutuellement prédictives (Sucre prédit à 3,80 pour Awa).

10. Réponse B — Un seul indicateur peut cacher la dégradation : le modèle prédit 9 admis pour 5 réels. La surveillance compare plusieurs métriques et les distributions, puis déclenche un réentraînement sur des données récentes étiquetées.

Résumé — les notions importantes

• Préparation : regarder, dédoublonner, séparer entraînement/test, puis seulement calculer médianes et ajuster les encodeurs — sur l'entraînement seul, sinon fuite de données.

• Arbre de décision : lisible mais vite surappris (profondeur 3 : 1,00/0,60) ; profondeur limitée (0,933/0,80) et forêt aléatoire (100 votes, importances mesurables) stabilisent les prédictions.

• Sur de petits jeux, un seul score ne prouve rien : la validation croisée compare honnêtement (forêt sur les étudiants : 0,70 ± 0,10).

• Réduire ou sélectionner les variables fait gagner en clarté : la PCA concentre 96,0 % de la variance corporelle dans une composante « taille générale » ; SelectKBest retient assiduité et exercices.

• Séries temporelles : évaluation toujours chronologique ; les références saisonnière (MAE 15,17), naïve (15,5) et moyenne mobile (22,89) sont les étalons que toute méthode avancée doit battre.

• Recommandation : similarité cosinus entre utilisateurs et moyenne pondérée des notes des voisins (Awa–Sucre prédit à 3,80/5).

• Déploiement : API validée et journalisée, puis surveillance multi-indicateurs — la dérive peut être conceptuelle (précision 0,556) même quand l'exactitude semble stable (0,80).

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 2 — Random Forest

Chaîne : Clémence Audrey · Langue : Français · Niveau : Intermédiaire

Le principe des forêts aléatoires expliqué simplement : combiner de nombreux arbres de décision pour obtenir une prédiction plus robuste qu'un arbre seul. À regarder après la leçon 2 pour consolider la notion de vote d'ensemble.

https://www.youtube.com/watch?v=QzPmJuYxOmg

Leçon 2 — La sagesse des forêts | Intelligence Artificielle 17

Chaîne : Science4All · Langue : Français · Niveau : Tous niveaux

Une vulgarisation soignée des forêts aléatoires et de la lutte contre la sur-interprétation des données. Elle éclaire le surapprentissage observé dans l'exemple du cours (train 1,00, test 0,60).

https://www.youtube.com/watch?v=D2IazsNG9_g

Leçon 4 — Prédiction de Séries Temporelles Avec Python et ARIMA : Guide Pratique Pour Débutants

Chaîne : LeCoinStat · Langue : Français · Niveau : Intermédiaire

La modélisation ARIMA pas à pas en Python : stationnarité, choix des paramètres, validation des résidus et prévision. C'est l'étape suivante naturelle après les méthodes de référence de la leçon 4.

https://www.youtube.com/watch?v=cwqdcNqRkyw

Leçon 6 — Exposer un modèle IA avec FastAPI : endpoints, Pydantic, le piège async

Chaîne : OUJOOD · Langue : Français · Niveau : Avancé

Créer une API de prédiction avec FastAPI : points d'accès, validation des entrées par Pydantic et documentation automatique. La vidéo mesure en direct un piège de performance lié à async, utile avant de déployer le modèle de la leçon 6.

https://www.youtube.com/watch?v=9uqaiLHPqGM

Leçon 6 — Flask vs FastAPI : Lequel Choisir pour Exposer ton Modèle IA en API ? (Tuto Complet 2026)

Chaîne : DatascienceMLopsClub · Langue : Français · Niveau : Intermédiaire

Un comparatif complet des deux frameworks Python pour exposer un modèle en API REST : création d'endpoints, consommation par requêtes, bonnes pratiques et sécurité. Il aide à choisir l'outil de déploiement de la leçon 6 en connaissance de cause.

https://www.youtube.com/watch?v=jnpR9ajYnoM

Sources et références

• Documentation officielle de scikit-learn (User Guide et exemples) — scikit-learn.org : la référence pour chaque classe utilisée dans ce cours (OneHotEncoder, DecisionTreeClassifier, RandomForest, PCA, SelectKBest, cosine_similarity).

• Efron, Hastie, Johnstone & Tibshirani — « Least Angle Regression », Annals of Statistics, 2004 : l'article source du jeu de données Diabetes intégré à scikit-learn, utilisé honnêtement comme données réelles dans la leçon 7.

• Géron, Aurélien — « Machine Learning avec Scikit-Learn » (Dunod) : ouvrage pratique de référence, avec un chapitre entier consacré aux fuites de données et aux pipelines.

• Hyndman & Athanasopoulos — « Forecasting : Principles and Practice », OTexts (libre accès en ligne) : la référence pour les séries temporelles, des méthodes naïves et saisonnières aux modèles avancés.

• Hastie, Tibshirani & Friedman — « The Elements of Statistical Learning », Springer (PDF libre sur le site des auteurs) : les fondements statistiques des arbres, des forêts aléatoires et de la sélection de variables.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !